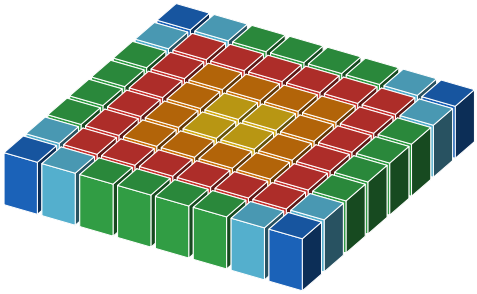
Test-Time Training on a Budget

ARC-AGI-2 on a Single Consumer GPU

Running the 2025 ARC Prize winning recipe, per-task test-time training with augment-infer-reverse-vote selection, on one 16 GB card

---



16.28%

on a single 16 GB GPU

up from 12.79%, 13× faster

ARC Prize 2026 · Paper Track

ARC Prize 2026 Paper Track entry

Open source under MIT-0

Abstract

The strongest published results on ARC-AGI-2 come from multi-GPU clusters. We ask how far the 2025 winning recipe, a small grid language model with per-task test-time training (TTT) and Augment-Infer-Reverse-Vote (AIRV) selection, goes on a single 16 GB consumer GPU, and where it walls out. Building on the released NVARC Qwen3-4B grid model, we report a reproducible offline system with a **12.79%** no-TTT baseline on the 120-task public evaluation set. We find that the binding constraint is generation throughput, not model quality or memory, and that the single largest lever is the inference engine, which buys a 13× decode speedup. We then show how to run per-task TTT on this hardware by serving each task's adapter on the fast engine, and report a measured lift: naive TTT ties the no-TTT baseline because the per-task adapter overfits, while a regularized, scale-tuned configuration raises accuracy from 12.8% to **17.6%** on the 84 trainable tasks, lifting the full evaluation to a blended **16.28%**. The contribution is an honest accounting of what the recipe costs and where the effort should go when you cannot rent a cluster. All code, the conversion recipe, and a development log of every measurement are open source under MIT-0.

Contents

1 Introduction **2**

1.1 Efficient abstraction is the point 2

1.2 Contributions 3

2 Related Work **3**

3 System and Method **4**

3.1 Model and prompting 4

3.2 The engine is the bottleneck 4

3.3 AIRV: augment, infer, reverse, vote 5

3.4 Per-task test-time training 6

4 Results **7**

4.1 No-TTT baseline 7

4.2 The engine speedup 7

4.3 Test-time training 8

4.4 Ablations: scale, colour, and the big-grid wall 9

5 Discussion **10**

5.1 Two lessons that generalize 10

5.2 Accuracy per GPU-hour 10

5.3 Limitations and ongoing work 10

6 Conclusion **11**

A Reproducibility **11**

B Extended results **12**

C Engineering log: walls and dead ends **12**

D Prompt and serialization format **13**

E Data and model provenance **13**

Reader's guide. Section 1 motivates efficient ARC and states the contributions. Section 2 places the work against the 2025 winning systems. Section 3 is the method: the model and prompt format (3.1), the engine bottleneck and its fix (3.2), the AIRV selection rule (3.3), and the per-task TTT bridge (3.4). Section 4 reports the numbers; Section 5 draws the lessons. Readers who want to reproduce the system should go straight to Appendix A; the per-scale tables and significance detail are in Appendix B; the honest list of dead ends is Appendix C; the exact prompt and 16-token vocabulary are in Appendix D; data licensing and BibTeX are in Appendix E.

1 Introduction

1.1 Efficient abstraction is the point

ARC-AGI-2 [1, 2] is a benchmark of abstraction and reasoning: each task gives a few input/output grid pairs that demonstrate a transformation, and the solver must apply that transformation to a held-out input. The tasks are designed to be easy for humans and hard for machines, and the competition is run *offline*, which rules out frontier API models and forces every entrant to ship a self-contained system. The 2025 ARC Prize was won by NVARC at 24.0% on ARC-AGI-2 using a 4B grid model with per-task TTT [3, 4]; the recipe reproduced across the top teams is the same: fine-tune a small model on augmented grid data, train a fresh low-rank adapter on each test task, then sample many augmented candidates and vote.

The recipe is effective but expensive: the public reports were produced on clusters of H100 or GH200 GPUs. Most researchers do not have that hardware, and the ARC Prize's stated mission is *efficient* skill acquisition, not skill bought with compute. We therefore characterize the recipe under a constraint we think is widely shared: a single RTX 5070 Ti with 16 GB, on the Blackwell architecture, running native Windows. The aim is not to top the leaderboard, which is unrealistic on one card, but to build a working, reproducible system, measure honest numbers, and map where one card stops being enough. Figure 1 shows a representative task our system solves end to end.

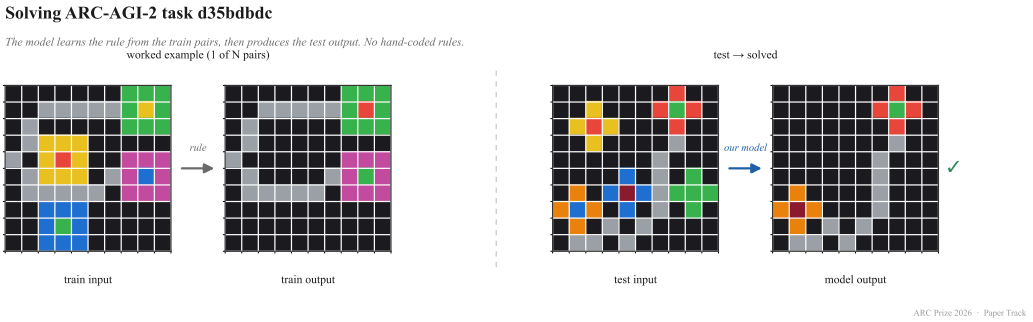


Figure 1: A real ARC-AGI-2 evaluation task solved by the no-TTT pipeline, shown as colour grids: the few-shot train pairs establish the rule, and the model produces the held-out test output exactly. Grids are rendered with the model's own 16-colour serialization (Appendix D).

1.2 Contributions

1. A reproducible offline ARC-AGI-2 system for the custom-vocabulary NVARC model on a single 16 GB card, with a clean **12.79%** no-TTT baseline (Section 4, Appendix A).
2. The finding that, on this hardware class, the inference *engine* dominates every other knob: moving decode off the Hugging Face eager path onto llama.cpp is worth a 13× speedup (Section 3.2).
3. A practical bridge that makes per-task TTT runnable on one card by hot-swapping GGUF adapters on the fast engine, and a measured, regularization-dependent lift to a **16.28%** blend (Section 3.4, 4.3).
4. An "accuracy per GPU-hour" accounting that names the three walls a compute-poor practitioner hits first: the decode ceiling, the VRAM limit on large-grid TTT, and adapter overfitting (Section 5).
5. A complete open-source release: the conversion patches, the pipeline, the TTT harness, and a development log of every measurement and dead end behind these numbers (Appendix C).

2 Related Work

ARC and the measure of intelligence. ARC was introduced to measure skill-acquisition efficiency rather than skill itself [1]; ARC-AGI-2 [2] raises compositional difficulty while keeping tasks human-solvable, widening the human/machine gap that frontier models had begun to close on ARC-AGI-1.

Test-time training. The dominant 2025 approach fits a small per-task adapter at inference, trading test-time compute for generalization. The ARC Prize 2025 technical report [3] identifies this, together with heavy synthetic pre-training and disciplined engineering, as the core of the winning NVARC solution [4], whose released model we build on directly. We hold this recipe fixed and ask what survives a single consumer GPU.

Augmentation and voting. Predicting under the eight dihedral symmetries of the grid, inverting each prediction back to the original frame, and voting is a standard self-consistency device for ARC; it is cheap, training-free, and provides a usable confidence signal (Section 3.3).

Synthetic data. Procedural generators such as RE-ARC [6] and LLM-mutated corpora supply the augmented grid data these models are trained on. We use RE-ARC only in an in-progress base-model probe (Section 5.3), not in the results below.

Efficient and on-device inference. Our engineering contribution, getting a custom-vocabulary grid transformer to decode near the memory-bandwidth roofline on a consumer card and then serving per-task adapters on it, sits in the practical lineage of quantized GGUF inference. The novelty is not the engine but the demonstration that the full TTT recipe, adapter serving included, fits one 16 GB card, and a measurement of exactly where it stops.

3 System and Method

3.1 Model and prompting

We use the NVARC `qwen3_4b_grids15_sft139` model, a Qwen3-4B [5] with a 16-token vocabulary: the ten grid colours as digits, a newline, two role markers, and three specials (Appendix D). A grid is serialized one token per cell, and a colour value equals its token id. Because the role and special tokens cannot be produced by tokenizing text, we assemble every prompt at the token-id level as a ChatML few-shot sequence; getting this byte-exact is the single most important correctness condition, since a one-token drift between training and inference silently destroys accuracy.

3.2 The engine is the bottleneck

Our most consequential finding concerns throughput. Decoding a 4B model is memory-bound: each token streams the full weight matrix from VRAM, so for a weight footprint M on a card of bandwidth B the decode rate is bounded by a first-order roofline

$$r_{\max} = \frac{B}{M}. \tag{1}$$

With $B = 896$ GB/s, Eq. (1) gives 132 tok/s in bf16 ($M = 6.77$ GB), 249 in Q8_0 ($M = 3.6$ GB), and 444 in Q4_K_M ($M = 2.02$ GB). The standard Hugging Face eager path prefills healthily ($\approx$ 3,400 tok/s) but *decodes* at only $\approx$ 13 tok/s, about a tenth of the bf16 roof, because of per-token framework overhead with no CUDA graphs. The usual remedy, compiling the decode step, is unavailable on this platform: `torch.compile` needs Triton, and Triton has no native-Windows backend (Appendix C).

The fix is to change engines. We convert the model to GGUF and serve it with llama.cpp. The custom vocabulary needs two small, documented converter patches (Appendix A): the unknown tokenizer hash is mapped to the `qwen2` pre-tokenizer it actually uses, and two throwaway byte-pair merges are added to satisfy a loader that requires a non-empty merge list (they never fire on grid text). The prebuilt CUDA 12.4 binary runs on the Blackwell card through PTX just-in-time compilation. Measured decode rises to 102 tok/s in bf16, 170 in Q8_0, and 244 in Q4_K_M (Figure 2), a 13$\times$ speedup over eager at Q8_0, which we adopt: it is near-lossless (14 of 15 training tasks reproduced exactly under greedy decoding) and has a 3.6 GB footprint. One stability note: the server's default prompt cache reserves $\approx$ 680 MiB per unique

prompt; since every ARC prompt is unique it yields no reuse and destabilizes sustained generation, so we disable it and run a single supervised stream.

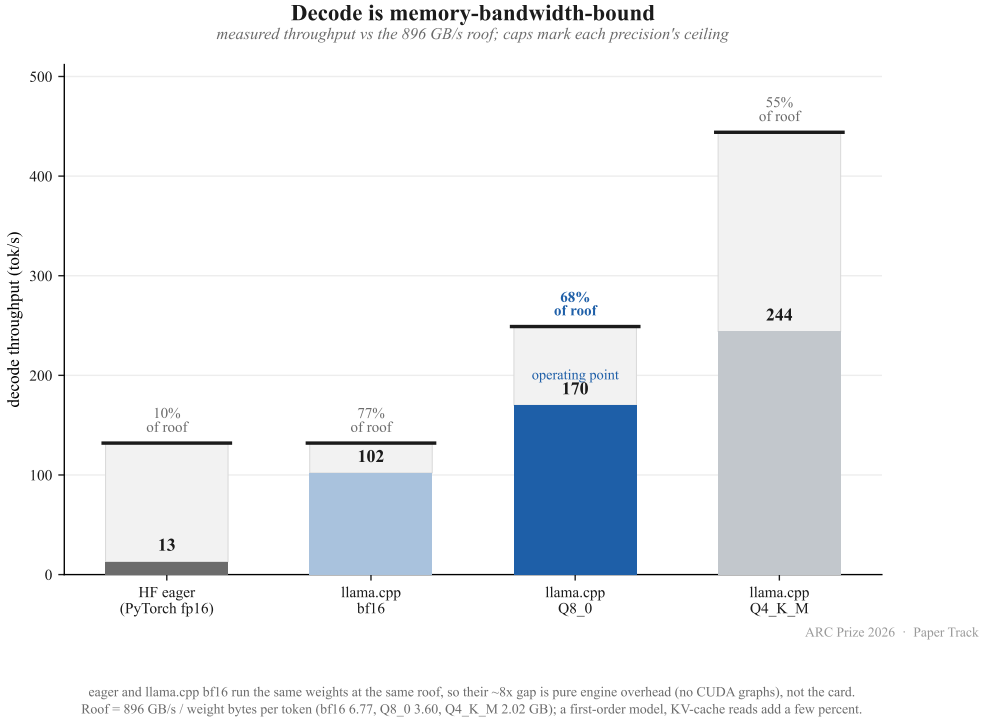


Figure 2: Decode throughput versus the memory-bandwidth roofline of Eq. (1) (the ceiling is a first-order model, bytes/token $\approx$ weight bytes). Bars are *measured* tok/s; the shaded gap to each cap is framework overhead, not the card. The eager path fills only $\approx 10\%$ of its roof; moving decode to llama.cpp at Q8_0 recovers the $13\times$.

3.3 AIRV: augment, infer, reverse, vote

Let f be the served model and $\mathcal{G}$ the eight dihedral (D4) symmetries of the grid. For a test input x we generate one prediction per symmetry, invert it back to the original frame, and vote. Writing n_c for a candidate's vote count and $\bar{\ell}_c$ for its mean sequence log-prob,

$$c_g = g^{-1}\big(f(g(x))\big), \qquad \hat{Y} = \operatorname*{select}_{c}\,\big(n_c,\,\bar{\ell}_c\big), \qquad n_c = \big|\{g \in \mathcal{G} : c_g = c\}\big|, \quad \bar{\ell}_c = \frac{1}{n_c}\sum_{g:\,c_g=c} \log p(c_g), \tag{2}$$

where select returns the two distinct candidates with the highest vote count, ties broken by mean log-prob, and emits them as the two attempts the metric allows. The vote count is itself a usable confidence signal: agreement across symmetries predicts correctness (Figure 6). The whole offline system (the trainer, the converter, and the server, which never share VRAM) is sketched in Figure 3.

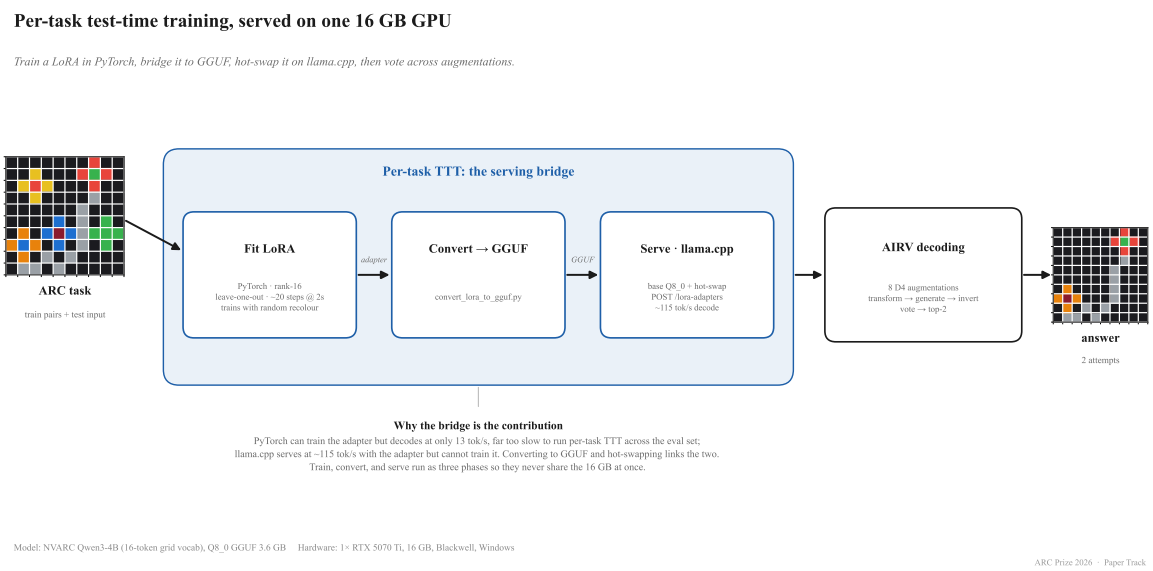


Figure 3: The offline system. The PyTorch trainer and the llama.cpp server never share VRAM; per-task adapters cross the bridge as GGUF and are hot-swapped on the fast engine per request.

3.4 Per-task test-time training

For TTT we fit a per-task LoRA adapter $\Delta W = sBA$ of rank r and scale s on the task's own train pairs T, with a leave-one-out target and the loss masked to the answer tokens:

$$\mathscr{L}(\theta) \;=\; -\sum_{(u,v)\in T}\;\sum_{t}\log p_\theta\big(v_t \mid v_{<t},\, \mathrm{ctx}_{T\setminus\{(u,v)\}},\, u\big), \tag{3}$$

drawing a fresh random augmentation each step. Capping the few-shot context to $\approx 1{,}600$ tokens drops a training step from ≈ 13 s to ≈ 2 s, so training is no longer the expensive part.

The harder problem is *serving* the trained adapter. TTT must run in PyTorch, where decoding is the slow 13 tok/s and intermittently becomes quadratic after a step; the fast engine does not train. We bridge them (Figure 3): train in PyTorch, save each adapter, convert it to a GGUF adapter, and hot-swap it on llama.cpp through its `/lora-adapters` endpoint, where decode holds ≈ 115 tok/s with no stalls. We run three phases (train all, convert all from a clean process, then serve and generate all) so the two frameworks never share VRAM (Figure 4). Large-grid tasks remain too VRAM-bound to train and are excluded by their largest-train-pair token length (Figure 4).

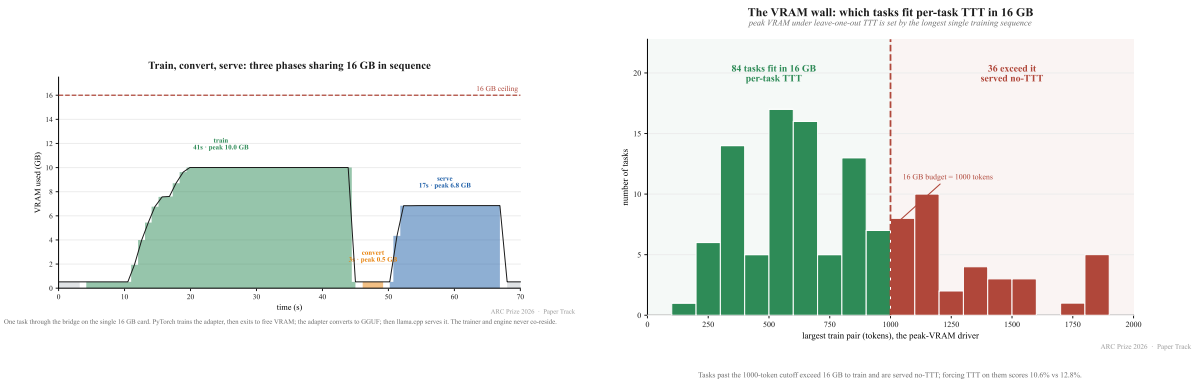


Figure 4: Left: the three-phase schedule keeps PyTorch and llama.cpp from sharing the 16 GB at once (VRAM trace sampled live; peaks measured). Right: the VRAM wall, with each task placed by grid size and context length; tasks above the operational cutoff are excluded from TTT (the cutoff metric is the largest train pair, and any VRAM surface shown is a calibrated model).

4 Results

4.1 No-TTT baseline

The AIRV pipeline without TTT scores **12.79%** (22 of 172 test outputs) on the 120-task public evaluation set, a clean single-stream run completing in $\approx$ 78 minutes. For reference the 2025 winner reached 24.0% with TTT [3].

4.2 The engine speedup

The 13$\times$ decode improvement from changing engines (Section 3.2, Figure 2) is the single most important result for this hardware class. It turned an overnight, frequently-failing process into a sub-90-minute evaluation. On one consumer GPU, the inference engine dominated every other knob we touched (Figure 5).

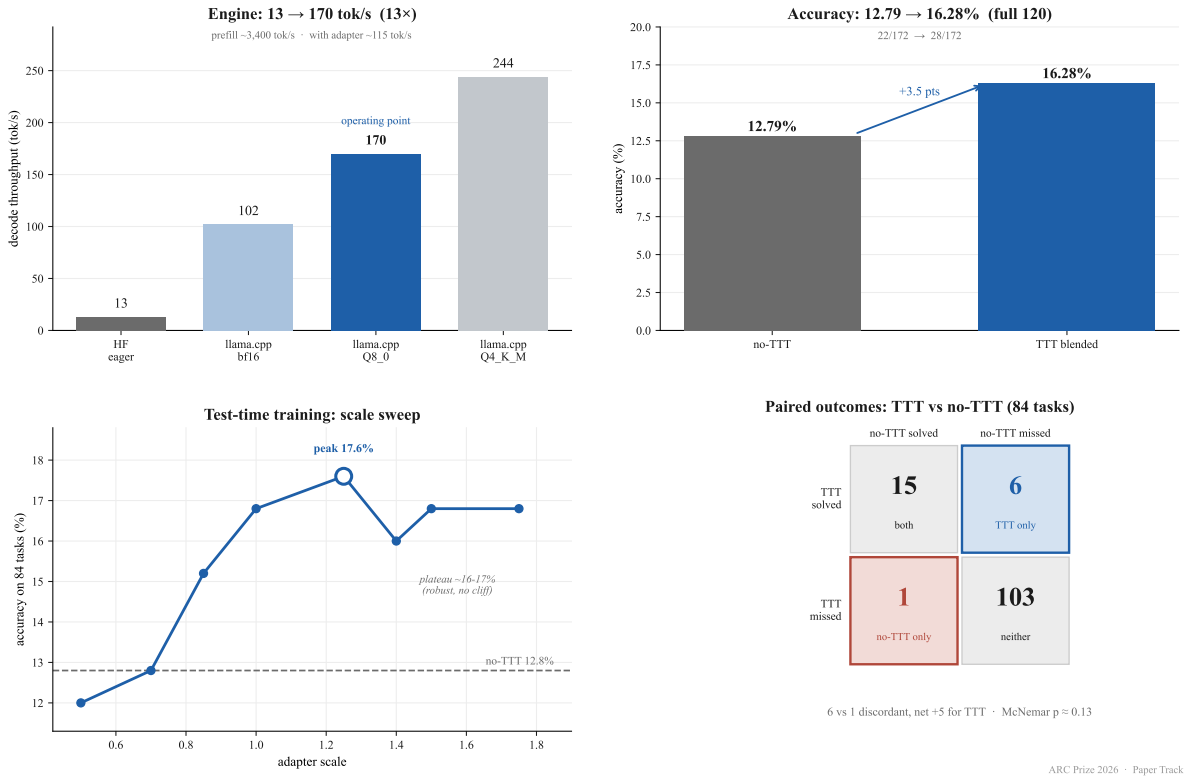


Figure 5: Engine choice, accuracy, and the adapter-scale curve. Accuracy peaks at scale 1.25 on the 84 trainable tasks, then declines, a real optimum rather than a monotone gain. All values are measured.

4.3 Test-time training

With generation fixed, we first swept the adapter configuration on 15 small-context tasks, scoring each against no-TTT on the same tasks (Table 1). The standard configuration overfits (its training loss reaches 0.000) and ties the baseline; the regularized configuration recovers a one-task lift; an over-aggressive reduction undertrains and ties again, coherent with an overfit diagnosis, though one task on 15 is only a hint.

Table 1: Adapter-configuration sweep on 15 small-context tasks (23 test outputs).

configuration	TTT	no-TTT
40 steps, rank 32, no dropout	3/23	3/23
20 steps, rank 16, dropout 0.1	**4/23**	3/23
8 steps, rank 8, dropout 0.05	3/23	3/23

We then validated the regularized configuration at scale, on the 84 evaluation tasks small enough to train on a 16 GB card, against no-TTT on the same tasks (Table 2). The lift holds and strengthens: +5 test outputs, a four-point gain ($\approx 31\%$ relative). The paired breakdown is clean: of the tasks the two methods

disagree on, TTT is right on six that no-TTT misses and wrong on only one. With seven disagreements this is not individually significant (exact McNemar $p \approx 0.13$, Appendix B), but three measurements now agree in direction (sweep $+1$, scale $+5$, six-to-one discordant), and no-TTT scores 12.8% here against its 12.79% full baseline, so the subset is representative.

Table 2: Regularized TTT vs. no-TTT on the 84 trainable tasks (125 test outputs), scale 1.0.

method	correct	accuracy
regularized TTT	**21/125**	**16.8%**
no-TTT	16/125	12.8%

4.4 Ablations: scale, colour, and the big-grid wall

Three further levers, each measured on the 84 trainable tasks (full per-scale numbers in Appendix B). The adapter *scale* matters: accuracy peaks at 17.6% at scale 1.25, then declines (Figures 5, 6). Colour permutation lifts the no-TTT baseline to 14.4% but does *not* stack with the already colour-robust adapter. Forcing TTT onto the big-grid tasks does not help; the VRAM exclusion is a genuine ceiling, not a missed opportunity. The best blend, TTT at scale 1.25 where trainable and no-TTT elsewhere, reaches **16.28%** on the full 120-task evaluation, up from 12.79%. The honest summary: naive per-task TTT *ties* a strong no-TTT AIRV baseline, while a regularized, scale-tuned TTT *beats* it by a consistent if modest margin.

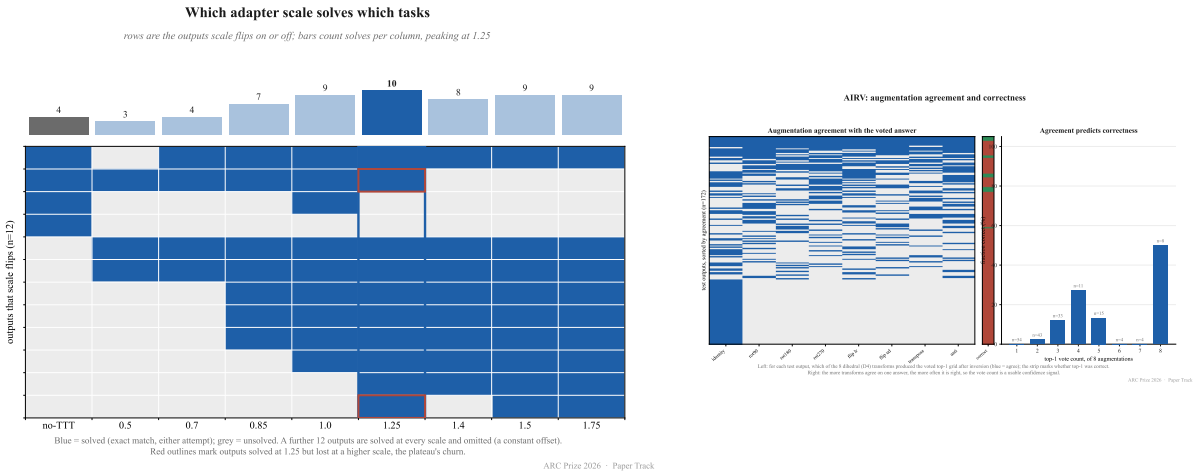


Figure 6: Left: which tasks TTT solves at each adapter scale, with most gains concentrated at scale 1.25. Right: AIRV vote agreement (Eq. (2)) predicts correctness; full eight-way consensus is right far more often than split votes.

5 Discussion

5.1 Two lessons that generalize

First, on a single GPU the inference engine is the dominant design choice; quantization and sampling mattered far less than the 13× we recovered by moving decode off the eager path. Second, the real obstacle for TTT is the *serving bridge*: training needs a gradient framework, fast generation needs an optimized engine, and per-task TTT produces a fresh adapter the engine must serve. We show this bridge is buildable on one card, and that once it exists the open question is not plumbing but *regularization*, because the small per-task training set overfits easily.

5.2 Accuracy per GPU-hour

We argue the right framing for compute-poor ARC work is **accuracy per GPU-hour** (Figure 7), and the map of walls reported here (the decode ceiling, the VRAM limit on large-grid TTT, and the overfitting of naive adapters; Figure 7) is a first step toward it. The findings tell a practitioner with similar hardware exactly which problems to solve first: fix the engine before touching anything else, budget VRAM by largest-train-pair length, and regularize the per-task adapter rather than training it harder.

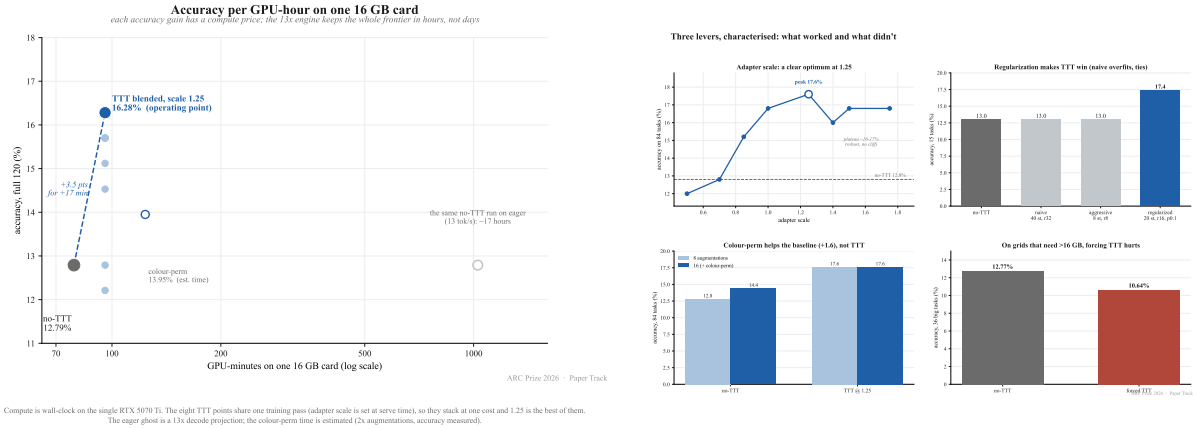


Figure 7: Left: accuracy per GPU-hour on one card; the TTT blend at 16.28% sits on the Pareto front (compute is measured wall-clock, and any eager "ghost" points are 13× projections, labelled in the source). Right: what worked and what did not. The engine swap and regularized TTT helped, while naive TTT, colour-perm stacking, and big-grid TTT did not.

5.3 Limitations and ongoing work

Our absolute accuracy is far below the 24.0% cluster result, by design: one card and a fixed recipe. The biggest known lever we still lack is verifier-guided selection and a refinement loop (generate, check each candidate against the train pairs, then re-prompt on the discrepancy), which the 2025 analysis names as the central advance; our AIRV is a one-shot vote. As ongoing work we are also probing whether a

one-time continued LoRA fine-tune of the base on official ARC-AGI-2 training data and RE-ARC [6] generations yields a better static model; that result is gated on beating the 12.79% baseline and is not claimed here. Finally, the significance of the TTT lift is directional rather than conclusive (Appendix B); a full-eval paired run is the obvious next measurement.

6 Conclusion

The 2025 winning recipe survives a single 16 GB consumer GPU, but only after the inference engine is fixed: that one change is worth more than every other knob, and it is what makes per-task TTT tractable through an adapter-serving bridge. The honest result is a 12.79% no-TTT baseline and a 16.28% regularized-TTT blend, with the gains bounded by adapter overfitting and a hard VRAM wall on large grids. The full system is open source: the GGUF conversion recipe for the custom-vocabulary model, the AIRV pipeline, the per-task TTT bridge and sweep harness, and a development log recording every measurement and dead end behind these numbers.

A Reproducibility

Environment. A single NVIDIA RTX 5070 Ti (16 GB, Blackwell, $\approx$ 896 GB/s), native Windows 11. Inference: a prebuilt llama.cpp CUDA 12.4 `llama-server`, run on Blackwell through PTX JIT. Training: PyTorch with a LoRA via PEFT, bf16 base, gradient checkpointing. The two frameworks are never co-resident in VRAM (Section 3.4).

The two GGUF converter patches. The NVARC model has a 16-token *WordLevel* tokenizer, which trips the stock converter twice. (1) *Unknown tokenizer hash:* the loader does not recognize the tokenizer's hash and refuses to guess a pre-tokenizer; its pre-tokenizer is in fact the `qwen2` regex, so we map the hash to `qwen2` explicitly. (2) *Empty merge list:* a WordLevel tokenizer has no byte-pair merges, but the loader requires a non-empty list, so we add two throwaway merges among the role tokens, which never fire on grid text. Both patches are in the repository's conversion script and are applied automatically by the Kaggle prep notebook.

Serving flags. `llama-server` is launched with full GPU offload (`-ngl 99`), a 16,384-token context, a single slot (`-np 1`), and the prompt cache disabled (`-no-cache-prompt`); its own logs are redirected to a file so they never flood the notebook. Decoding is greedy (temperature 0) with `n_predict` sized to the target grid.

Hyperparameters. AIRV uses the eight D4 augmentations and votes to two attempts. Per-task TTT (regularized): rank 16 to 32, dropout 0.05 to 0.1, 8 to 20 steps, AdamW, a fresh random augmentation per step, leave-one-out target, context capped at $\approx$ 1,600 tokens, applied at adapter scale 1.25 at inference. Exact values per experiment are in the development log.

B Extended results

Per-scale accuracy (84 trainable tasks). Table 3 gives the full adapter-scale sweep behind Figures 5 and 6. Accuracy rises to a peak at scale 1.25 and then declines, the signature of an adapter applied too strongly.

Table 3: No-TTT vs. TTT accuracy on the 84 trainable tasks as a function of adapter scale.

adapter scale	no-TTT	0.50	0.70	0.85	1.00	**1.25**	1.40	1.50	1.75
accuracy (%)	12.8	12.0	12.8	15.2	16.8	**17.6**	16.0	16.8	16.8

Paired significance. On the 84 trainable tasks, TTT and no-TTT disagree on seven test outputs: TTT is correct on six that no-TTT misses and incorrect on one that no-TTT gets. An exact two-sided McNemar test on a $6/1$ split gives $p \approx 0.13$, so the lift is not individually significant; we report it as directional, corroborated by the configuration sweep ($+1$) and the representativeness check (no-TTT scores 12.8% on the subset versus 12.79% on the full set).

Colour permutation. Adding a random colour permutation to the augmentation set lifts the no-TTT baseline from 12.8% to 14.4% on the 84 tasks, but provides no further gain on top of the per-task adapter, which has already learned the task's colour mapping, so the two do not stack.

C Engineering log: walls and dead ends

We record the failures explicitly, because on this hardware they are most of the work.

- **Eager decode is overhead-bound, and the usual fix is unavailable.** Hugging Face eager decode runs at ≈ 13 tok/s, about 10% of the bandwidth roofline. `torch.compile` would close the gap, but it needs Triton, which has no native-Windows backend. The resolution was to change engines, not to optimize the eager path.
- **The server prompt cache destabilizes unique-prompt workloads.** Enabled by default, it reserves ≈ 680 MiB per unique prompt; every ARC prompt is unique, so it never reuses and instead destabilizes sustained generation. Disabling it and running a single supervised stream fixed the stalls.
- **4-bit QLoRA is broken on Blackwell + Windows.** bitsandbytes fails to load on this platform, so the planned 4-bit continued fine-tune was infeasible. Because per-task TTT already runs in bf16 inside 16 GB, we use a bf16 LoRA throughout and drop the 4-bit path entirely.
- **PyTorch decode can turn quadratic after a training step.** Generation in the gradient framework is not only slow but intermittently degrades after an optimizer step; this is precisely why the serving bridge exists, since we never generate in PyTorch in the scored pipeline.
- **The VRAM wall on large grids is real.** The largest-train-pair token length, not the test grid size, is

the binding constraint; tasks above the operational cutoff cannot train in 16 GB. Forcing TTT onto them by truncation did not recover accuracy, so they are excluded and served no-TTT.

- **Naive TTT ties the baseline.** The standard adapter configuration drives training loss to 0.000 and generalizes no better than no-TTT; only regularization (fewer steps, lower rank, dropout) recovers a lift.

D Prompt and serialization format

The model's 16-token vocabulary is given in Table 4. A grid is serialized row by row, one token per cell (the token id equals the colour), rows separated by the newline token; prompts are assembled at the id level as a ChatML few-shot sequence of train pairs followed by the test input, ending at the `ASSISTANT` turn for the model to complete. Because the role and special tokens have no textual spelling, the whole sequence is built from ids directly, never by tokenizing a string, which is what guarantees parity between training and inference.

Table 4: The 16-token grid vocabulary.

token ids	meaning
0 to 9	the ten ARC grid colours (id = colour)
10	newline / row separator
11	`USER` role marker
12	`ASSISTANT` role marker
13	`PAD`
14	`IM_START`
15	`IM_END`

E Data and model provenance

The base model is Qwen3-4B [5] (Apache-2.0), supervised-fine-tuned on ARC grids by NVARC [4]; any derived weights we redistribute are released under Apache-2.0 with the required `NOTICE`. Evaluation and training tasks are the official ARC-AGI-2 corpus [2] (Apache-2.0); the in-progress fine-tune additionally uses RE-ARC [6] (MIT). Our own code and weights are released under MIT-0. We never train on the evaluation tasks. Full license texts and attribution are in the repository's `THIRD_PARTY_LICENSES.md` and `NOTICE`.

BibTeX.

```
@article{chollet2019measure,
  title={On the Measure of Intelligence},
  author={Chollet, Francois}, journal={arXiv:1911.01547}, year={2019}}
@techreport{arcagi2,
```

```
  title={ARC-AGI-2 Technical Report},
  institution={ARC Prize Foundation}, year={2025}}
@techreport{arcprize2025,
  title={ARC Prize 2025: Technical Report},
  institution={ARC Prize Foundation}, note={arXiv:2601.10904}, year={2026}}
@misc{qwen3, title={Qwen3 Technical Report}, author={Qwen Team}, year={2025}}
@misc{nvarc2025,
  title={NVARC: ARC Prize 2025 (ARC-AGI-2) 1st-place solution},
  author={{NVIDIA Kaggle Grandmasters}}, year={2025}}
@article{hodel2024rearc,
  title={Addressing the Abstraction and Reasoning Corpus via
         Procedural Example Generation},
  author={Hodel, Michael}, journal={arXiv:2404.07353}, year={2024}}
```

References

[1] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[2] ARC Prize Foundation. ARC-AGI-2 Technical Report. 2025. https://arcprize.org.

[3] ARC Prize Foundation. ARC Prize 2025: Technical Report. arXiv:2601.10904, 2026.

[4] NVIDIA Kaggle Grandmasters (NVARC). ARC Prize 2025 (ARC-AGI-2) 1st-place solution. 2025.

[5] Qwen Team. Qwen3 Technical Report. 2025.

[6] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024.